Development and Evaluation of a Cryptocurrency Market Prediction Pipeline

Team 23 Members: Nikhar Arora, Nikoloz Jaghiashvili

Abstract

We present a reproducible, experimentally driven pipeline for forecasting Bitcoin price movements at minute resolution. The pipeline combines disciplined pre-model diagnostics, including pairwise correlation filtering, mutual information, and variance inflation factor analysis. Representation learning is applied through a supervised autoencoder that compresses predictors into a 64-dimensional latent space, followed by augmented recursive feature elimination that reduces the feature set to 50 variables. We benchmarked boosted tree models and multilayer perceptrons (MLPs). MLPs demonstrated superior validation stability, reduced overfitting, and tighter hyperparameter sensitivity profiles. Hyperparameter landscape analysis confirmed the presence of broad, stable regions rather than narrow optima, which indicates robustness to parameter shifts. The entire workflow was tracked with MLflow for full reproducibility. We document statistical motivations, quantify empirical gains, and provide recommendations for further research and operationalisation.

1. Introduction

Cryptocurrency markets at high frequency are noisy, nonstationary and subject to abrupt regime shifts. Effective supervised learning in this domain therefore requires a principled experimental workflow that (1) diagnoses and mitigates covariate problems (redundancy, drift, leakage), (2) extracts stable representations, (3) evaluates under time-aware protocols that preserve causality, and (4) records experiments to ensure reproducibility. Our implementation follows these principles across three phases: (A) feature selection and diagnostics, (B) model refinement and representation learning, (C) competition-style final training and submission. This document synthesizes the reasoning embedded in the codebase and the empirical outcomes logged in MLflow.

2. Methods

2.1 Data and experimental setup

Target: Anonymized cryptocurrency price movement, with an unknown transformation applied to it. Based on our exploratory analysis, the target isa scaled version of the Bitcoin price movement.

Figure 1: Cumulative Target vs BTC Price

Features: A set of 5 known order book features coupled with 780 anonymized features.

- **timestamp:** The timestamp index representing the minute associated with each row.
- **bid_qty:** The total quantity buyers are willing to purchase at the best (highest) bid price at the given timestamp.
- **ask_qty:** The total quantity sellers are offering to sell at the best (lowest) ask price at the given timestamp.
- **buy_qty:** The total trading quantity executed at the best ask price during the given minute.
- **sell_qty:** The total trading quantity executed at the best bid price during the given minute.
- **volume:** The total traded volume during the minute.
- **X_{1,...,780}:** A set of anonymized market features derived from proprietary data sources.

Training data: The train set contains 525,886 minute-level observations between March 1, 2023, to February 29, 2024

Test data: The test data comes after the training period and contains years' worth of minute level observations between March 1, 2024, to February 29, 2025. The timestamps in the test data are anonymized and shuffled, which prevents direct use of time-series techniques during modeling.

Environment: The code is designed to run in Kaggle notebook with GPU accelerator enabled and DRW dataset imported. Each notebook contains code to pull the Github repository and install all of the required dependencies. The repository uses a set of custom functions to pre-compute metadata tables, and MLflow to log the experiment results. In case when metadata is re-computed on new experiment, *github_push_file.py* and *github_push_folder.py* are used to push the changes to remote repositories.

Evaluation protocol: The competition is evaluated using pearson correlation coefficient between the labels and predicted values. The test set labels are not published after the end of competition. For internal evaluation extended set of parameters is used and captured by Mlflow, coupled with cross-validation using both simple and purge time-series splits.

2.2 Feature Selection

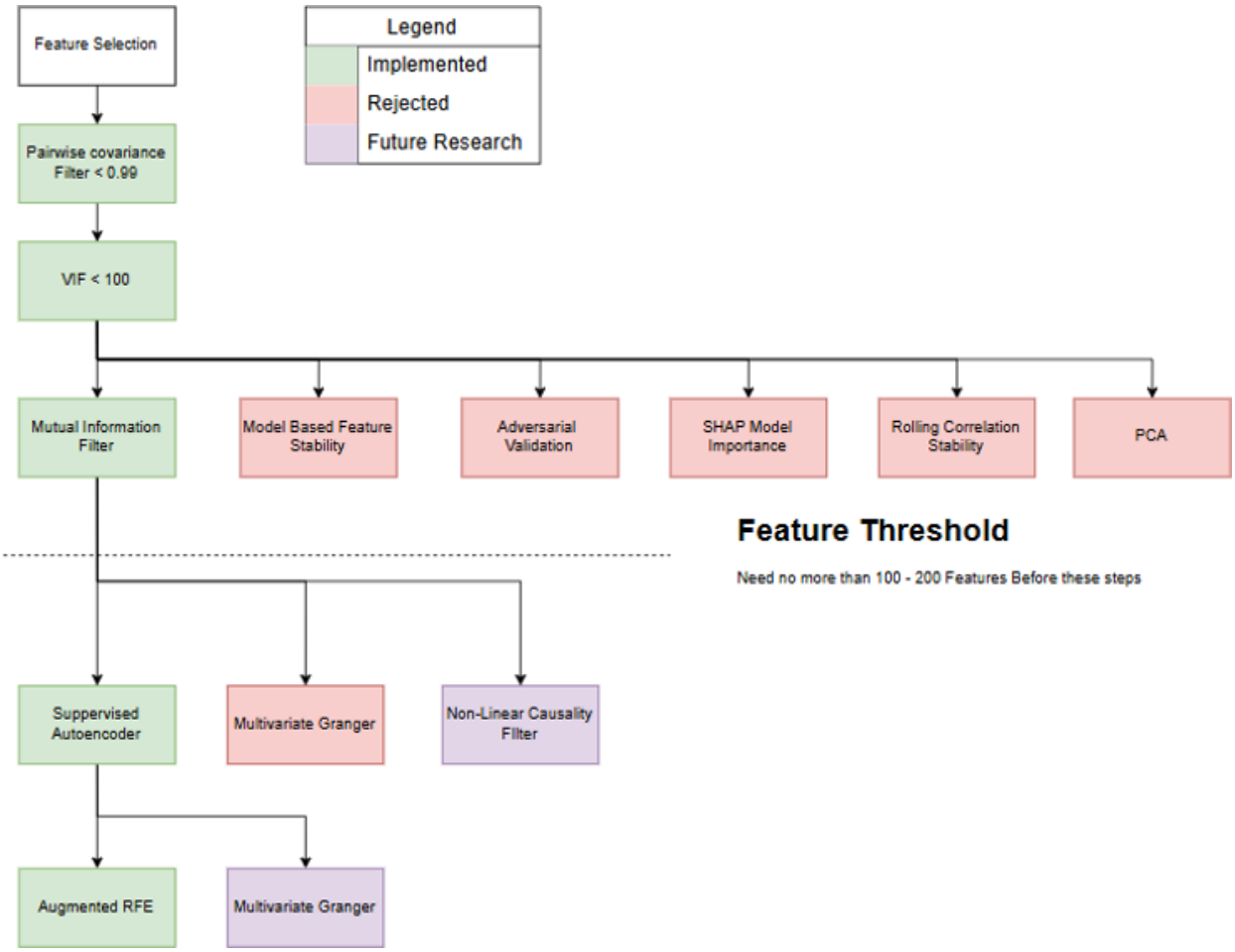
Given the large size of the dataset, the first goal in feature selection was to shrink the feature set using computationally inexpensive methods with low risk of information loss. To successfully apply more complex feature filtering or transformation approaches, the initial filter needed to reduce the feature space to roughly 100–200 features (depending on the downstream method). Given these requirements, conservative correlation and multicollinearity filters were great candidates for initial filters. Although a VIF filter may discard features that participate in nonlinear relationships, a relatively high threshold mitigates this risk.

Since conservative pairwise correlation and VIF filters reduced the features to 335, we required an additional filter with a low risk of information loss. We deliberately avoided PCA-like transformations to prevent potential nonlinear information loss. As a straightforward next step, we applied a conservative mutual information filter. We also tested other initial filters, including correlation stability, adversarial selection,

and model-based importance. However, these either risked losing too much information, introduced data leakage, or produced subpar results.

To aid experimentation, we implemented the get_feature_set() function, which produces a set of features based on various initial-filter configurations. The entire logic of feature selection can be viewed in Figure X.

Figure 2: Feature Engineering Diagram



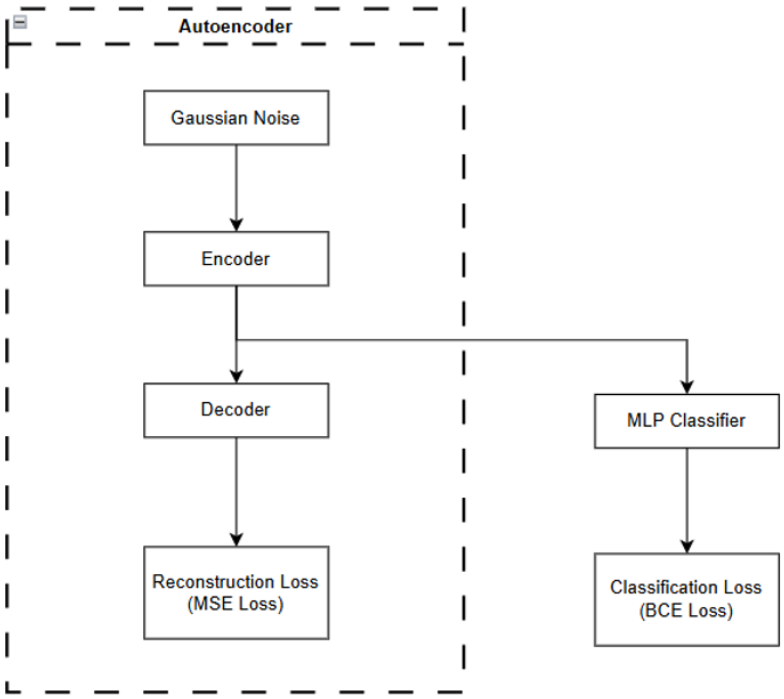
As part of our downstream feature engineering, we initially applied multivariate Granger causality for feature selection, following the idea that stable causality leads to stable relationships (Li et al., 2024). Additionally, we implemented an augmented version of recursive feature elimination, which re-fits the model and performs cross-validation after eliminating the least important feature, restoring it if performance does not degrade. While conceptually sound, the approach did not yield favourable results, resulting in a private test set performance of 0.062. Future work could test nonlinear approaches such as Neural Granger or DynoTears, or apply Granger selection to denoised data from our autoencoder.

The augmented RFE uses model-based feature elimination with a tuned LightGBM as the base model, chosen for its speed and accuracy on large tabular datasets. At each step, the model is retrained, the least important feature is dropped, and the impact is checked with cross-validation, restoring the feature if performance declines.

2.3 Representation learning and feature reduction

Our final approach employed a supervised autoencoder to encode features into a lower-dimensional latent space (Figure X). The encoder is trained jointly with an MLP classifier, which uses the latent representation to predict the direction of price movement. The objective is to obtain a denoised latent representation that retains predictive power, achieved by minimizing a weighted sum of the reconstruction loss and the classification loss. This approach, inspired by Bieganowski and Ślepaczuk (2024), which was also used by the following submission in Jane Street Competition. The goal is to mitigate noise and overfitting in high-dimensional financial data.
In the Jane Street competition, the authors noted variability in results across random seeds and addressed this by ensembling predictions from multiple runs. In our experiments, we found that increasing the weight on the reconstruction loss achieved comparable stability without degrading performance.

Flowchart 1: Supervised Autoencoder Diagram



Purged time series cross-validation was applied to minimise data leakage, as financial data inherently carries forward into future observations. Performing our original

augmented RFE approach on 64 latent features proved even more effective in latent space, further shrinking feature spaces to 50 features, resulting in a moderate boost in performance. This yielded a final score of 0.082, marking a significant improvement over the original causality based approach.

2.4 Models and training

Training period. Based on consistent degradation in the most recent (5th) fold, when performing cross-validation across all models and feature sets, and a pronounced structural break around November 2023 (Figure X), we restricted training to March 1–October 31, 2023. The same window was used for the supervised autoencoder. As future research, window selection could be formalized via change-point detection and regime clustering to automatically identify higher-performing training periods.

Tree based models: Tree-based models were implemented using LightGBM and XGBoost, with hyperparameters such as maximum depth, learning rate, and L1/L2 regularization tuned for optimal performance. These models were evaluated using simple time series cross-validation, with performance measured by metrics including Pearson correlation, mean squared error (MSE), and the overfitting gap, given the inherently noisy nature of time series data. Despite the applied regularization, the tree-based models showed noticeably greater overfitting compared to the MLP with weight decay regularization.

MLP: We used a PyTorch MLP with two hidden layers (ReLU + BatchNorm and dropout). Hyperparameters tuned included hidden sizes (e.g., 64–128, 128–256), learning rate, weight decay, and dropout. Evaluation used regular k-fold time-series CV (no purge), with Pearson correlation, MSE, and an overfitting gap between train/validation. With weight decay and dropout regularization, the MLP controlled overfitting well and delivered stronger validation correlation than the tree baselines.

3. Results

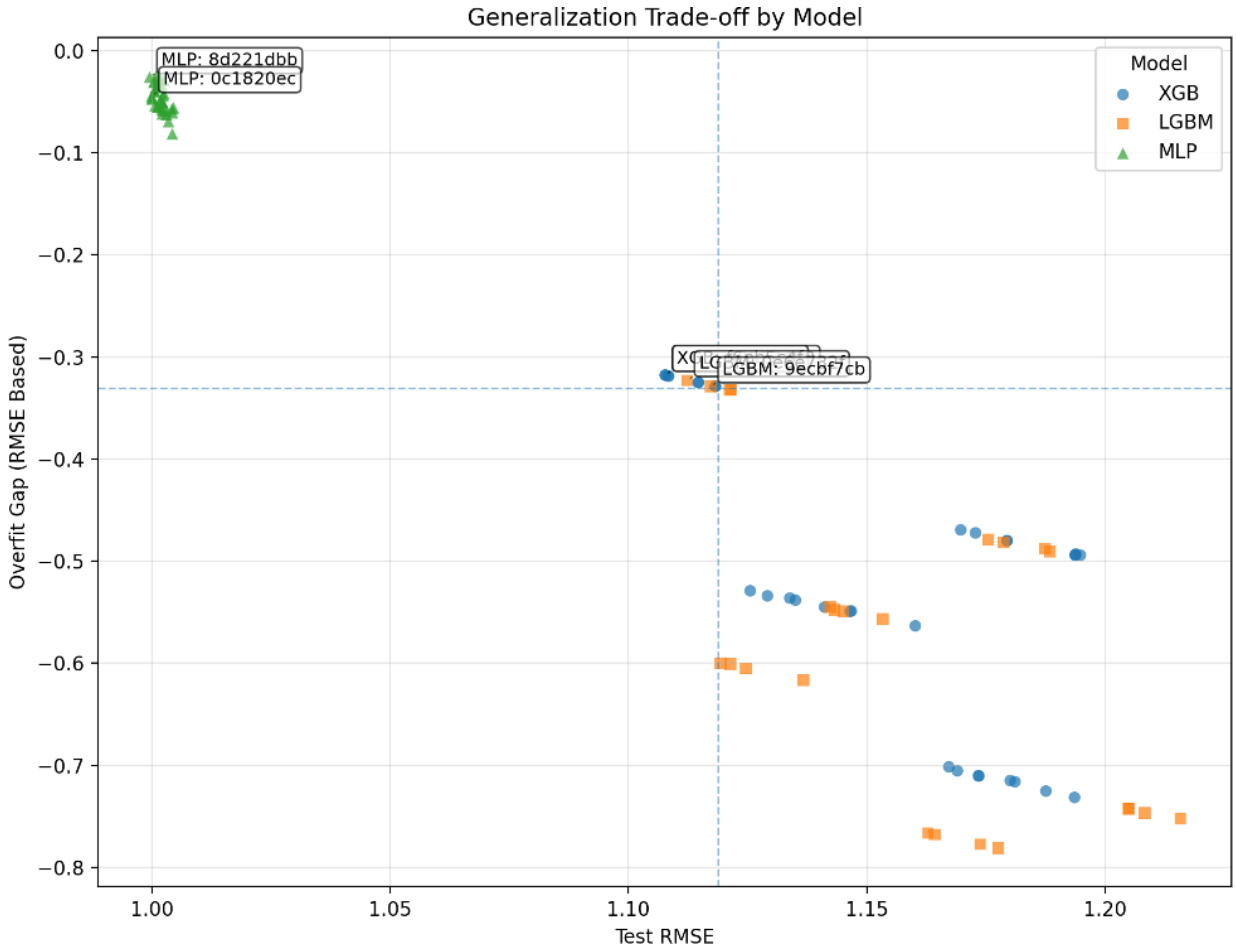
3.1 Feature selection outcomes

- **Correlation filtering:** >20 near-identical pairs (|corr|>0.99) removed.
- **VIF filtering**: iterative feature elimination; retained ≈ 336 predictors.
- **Mutual Information filtering:** reducing feature space to ≈ 236 predictors.
- **Supervised Autoencoder:** project 236 features into 64 latent feature space.
- **Augmented RFE:** Further reduce 64 latent features to final 50 predictors.

3.2 Model refinement and representation learning

During our experiments, MLP regression showed consistently better accuracy and a lower rate of overfitting. Furthermore, based on Figure 3, MLP is significantly more tightly clustered compared to tree-based models, demonstrating higher stability and lower hyperparameter sensitivity.

Figure 3: Hyperparameter Tuning. CV Performance vs Overfitting Gap



Run ID	Model	RMSE	Overfit Gap	Learning Rate	Reg Lambda	Weight Decay
f6eb5c4f	XGB	1.11	-0.32	0.5	0.5	
0e6e732f	LGBM	1.11	-0.32	0.5	0.5	
9ecbf7cb	LGBM	1.12	-0.33	0.5	0.1	
0c1820ec	MLP	1.0	-0.04	0.005		0.08
8d221dbb	MLP	1.0	-0.03	0.01		0.05

As seen in the table above, both MLP runs achieved the lowest RMSE (1.0) with minimal overfit gaps (–0.04 and –0.03), compared to tree-based models such as XGB and LGBM that reported higher RMSE (~1.11–1.12) and much larger overfit gaps (–0.32 to –0.33).

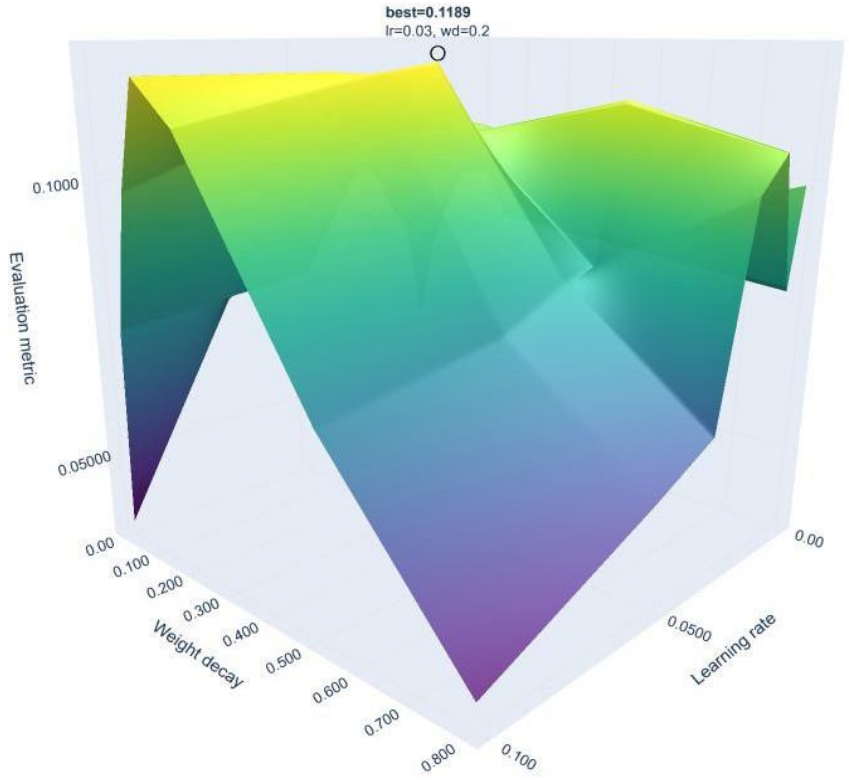
3.3 Competition runs

For the final competition submission, we used a tuned **MLP** model with two hidden layers of sizes **128** and **256**, a learning rate of **0.005**, a dropout rate of **0.3**, and a weight decay of **0.1** to enhance generalisation. The final competition scores recorded were a Pearson correlation coefficient of 0.082, which is within the top 10% of the final leaderboard.

3.4 Hyperparameter Landscape Analysis

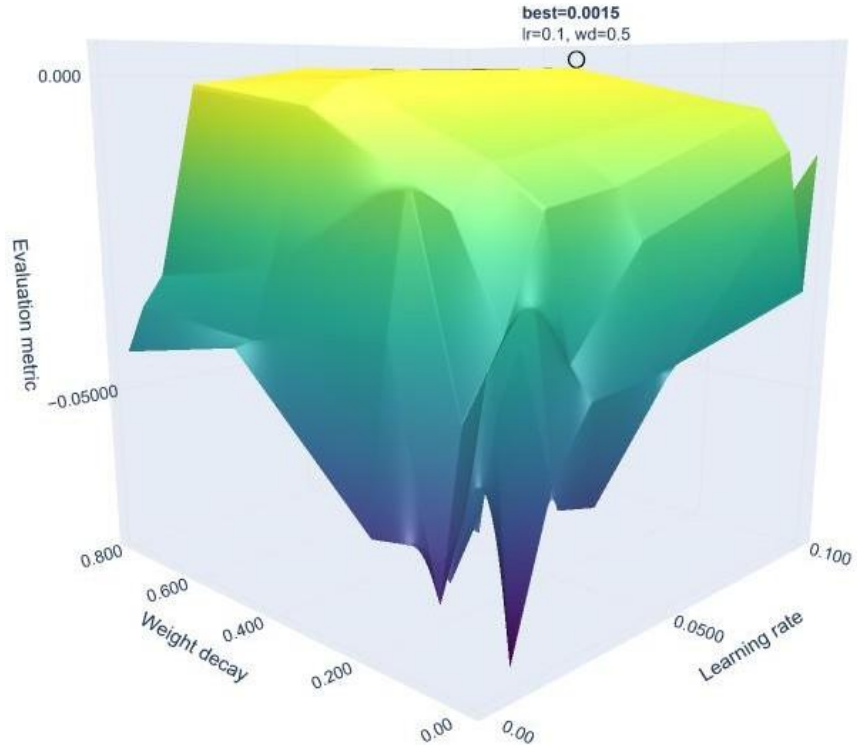
To further assess model stability and robustness, we performed a targeted hyperparameter sensitivity analysis beyond the initial grid search. In this extended exploration (Figure 4), we systematically varied the learning rate and weight decay across a denser grid of values. This experiment can be reproduced by running: This can be replicated by running run_cv.run_grid_search() with mode="sensitivity_analysis".

Figure 4: MLP Sensitivity Analysis - RMSE



Increasing weight decay initially acts as a strong regularizer, reducing overfitting and improving generalization. However, beyond a threshold, excessive regularization suppresses the model's capacity to learn complex patterns, leading to a decline in performance(in figure 4). Learning rate primarily controls convergence dynamics. Very low values slow training and risk underfitting, while excessively high values destabilize learning. The surface reveals an optimal band where the learning rate is high enough to ensure efficient convergence but low enough to maintain stability. The prominent peak at lr=0.03, wd=0.2 corresponds to the globally optimal setting, achieving an evaluation metric of 0.1189.

Figure 5: MLP Sensitivity Analysis - Overfitting Gap



The overfitting sensitivity surface in Figure 5 highlights how learning rate and weight decay jointly affect model generalization. Weight decay initially improves generalization by constraining model complexity, but excessive values reduce capacity and lower the overfitting score. Similarly, very low learning rates tend to underfit, while overly high rates destabilize training. The surface reveals a broad plateau of stable configurations rather than a sharp peak, suggesting that MLP performance remains robust across a range of balanced parameter values, mitigating the risk of over-tuning.

4 Limitations and failure modes

• **Nonstationarity**: models trained on historical data require continuous monitoring and periodic retraining as regimes change.
• **Over-pruning risk**: automated selection can remove causal features; maintain a small domain whitelist to prevent accidental exclusion.
• **Interpretability**: latent representations reduce direct interpretability; employ SHAP and VSN logs for post-hoc explanations.

The final model demonstrated robust validation performance with reduced overfitting, supported by latent-space feature selection and weight decay regularization. Immediate next steps include: (1) expanding nonlinear causality filters for denoised features, (2) implementing walk-forward validation to better capture temporal stability, (3) formalizing training-window selection via regime detection, and (4) extending our early experiments with conformal change point detection, which have shown initial promise. Specifically, we propose integrating Conformal CPD alerts into the production pipeline both as retraining triggers and as execution-gating signals to prevent trades during high-uncertainty regimes. These refinements aim to further harden the pipeline for production deployment and enhance competitive performance in evolving market conditions.

Beyond the Competition

We present a reproducible, research-oriented study of a Temporal-Fusion-inspired transformer (the **Momentum Transformer**) designed to forecast a continuous market label on anonymized minute-level cryptocurrency data (DRW competition). The pipeline combines disciplined preprocessing (sample-based column selection, VWAP, MACD, RSI, Bollinger Bands), a CUSUM-based changepoint detector (CPD), a Variable Selection Network (VSN) for per-sample feature gating, and a causal-attention transformer with LSTM encoding. Evaluated on a time-aware 20/80 chronological split, the complete pipeline attains **weighted Pearson ≈ 0.905–0.907** and **unweighted Pearson ≈ 0.8191** on held-out validation. Diagnostics (residual histogram, residuals vs predicted, learning curves, predicted vs actual) show generally centered errors with leptokurtic tails, slight heteroskedasticity in extremes, and stable convergence under gradient clipping and early stopping. We discuss methodological trade-offs (including a deliberate 20/80 split deviation), failure modes, and a prioritized research and production roadmap that emphasizes robustness, uncertainty quantification, and governance.

2. Methods

2.1 Data and computational setup

- **Dataset:** DRW Crypto Market Prediction (Kaggle) — anonymized minute-level covariates (up to 872 features) and continuous label.
- **Environment:** Python (Kaggle-like runtime), PyArrow/Parquet for columnar IO, PyTorch for deep models, LightGBM for baselines, and MLflow for experiment tracking.
- **Validation protocol:** a time-respecting **20/80 chronological split** was used for research validation: earliest 80% for training and internal CV, latest 20% reserved as held-out validation. This preserves causality for sequence models and gives realistic forward-looking estimates.

2.2 Memory-efficient column selection and label sanitization

- A seeded random sample (n=5,000) computed Pearson correlations with the label and identified candidate features with $|\text{corr}| > 0.05$ to reduce full-table I/O.
- Full data was loaded only for selected columns to avoid OOM errors.
- Label cleaning: infinities and extreme outliers were filtered (kept $|\text{label}|$ within $[0.002, 10]$); training used label-absolute weights for the weighted Pearson metric, emphasizing high-impact events.

2.3 Feature engineering

- **Technical indicators:** VWAP (volume-weighted average price), momentum windows (5, 10, 21), MACD (EMA differences), RSI(14), Bollinger Bands (20-period), and targeted nonlinear terms (top correlated features squared).

- **CUSUM CPD:** applied to VWAP to produce a severity score normalized to [0,1]; thresholded (≈3.0 × rolling std) to flag abrupt regime changes. CPD features were fed directly into VSN and attention inputs.
- **Sanitization:** constant and near-constant cols removed; near-perfectly correlated pairs (r>0.99) pruned. Resulting working set: a compact mixture of ~20–30 raw candidates plus engineered indicators and CPD.

2.4 Model architecture: Momentum Transformer

- **Input:** per-timestep feature projection to d_model = 32.
- **VSN (Variable Selection Network):** sample-wise gating producing per-feature weights and an interpretable importance signal; also reduces noise input to the encoder.
- **Temporal encoder:** LSTM for local sequence encoding (tested seq_len = 5 and seq_len = 21), followed by causal multi-head self-attention (4 heads, triangular mask).
- **Gating/FFN:** Gated Linear Units (GLU) and Gated Residual Networks (GRN) to provide adaptive nonlinearity and stable gradients.
- **Loss:** hybrid objective 0.7*MSE + 0.3*(-Sharpe_like_reward) to bias toward risk-adjusted, directional usefulness.
- **Stabilization:** gradient clipping (norm = 1.0), LR scheduling (ReduceLROnPlateau), early stopping on validation weighted Pearson (patience = 5), and gradient accumulation to manage memory.

2.5 Training regimen, baselines and experiment tracking

- **Optimizer:** Adam with lr ≈ 1e−3 (initial), batch sizes tuned to memory constraints, up to 50 epochs.
- **Baselines:** Linear models, LSTMs, LightGBM, and MLPs (1–3 layers). LSTMs improved over linear but were computationally heavy. Transformers outperformed in capturing interactions after incorporating VSN and CPD.
- **MLflow:** all hyperparameters, train/val losses, model checkpoints, feature lists, and VSN weights were logged for reproducibility.

3. Results

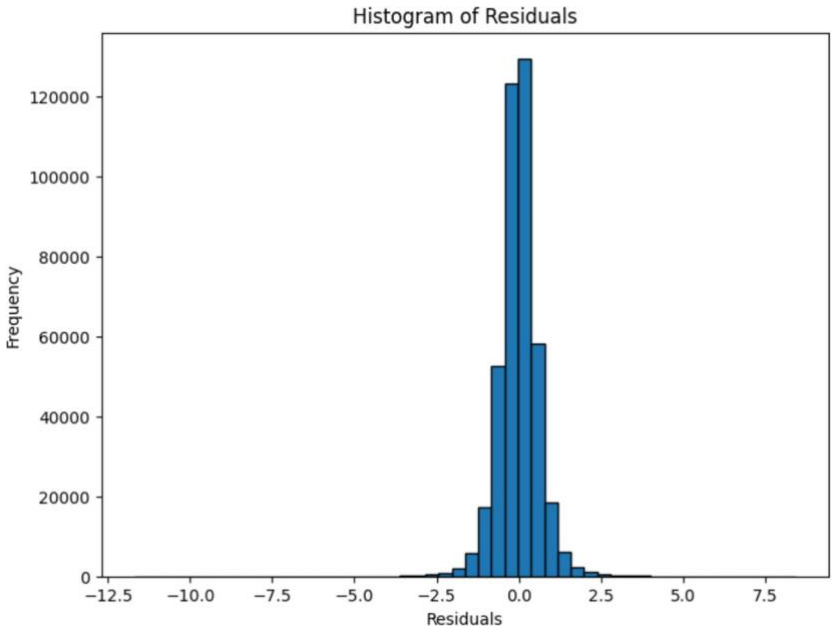
3.1 Primary validation metrics

- **Weighted Pearson (primary metric):** ≈ **0.905-0.907** on held-out chronological validation (weights = |label|).
- **Unweighted Pearson:** ≈ **0.8191** (representative run).
- **Training/validation MSE (representative):** train final ≈ **0.33**, val final ≈ **0.36** (converged by epoch ~30–50).
- **Residual std:** ≈ √(0.36) ≈ **0.6** (consistent with MSE).

The provided graphs stem from the validation phase of a Momentum Transformer model implemented in Python for cryptocurrency market label prediction. This model, a simplified transformer with input projection, position-wise feed-forward networks,

and sequence flattening, is trained on sequenced data (length=5) from scaled features including VWAP, momentum, and changepoint indicators. Using a 20/80 train-validation split (104,835/419,342 sequences), it optimizes MSE loss over 50 epochs with Adam (lr=0.001) and early stopping. Denormalized predictions (val_preds = val_outputs * y_std + y_mean) enable these diagnostics, revealing a final validation Pearson correlation of 0.8191 and weighted Pearson of 0.9051. Below, I analyze each graph, linking to code elements like residual computation (residuals = y_val_denorm - val_preds) and loss tracking (train_losses, val_losses).

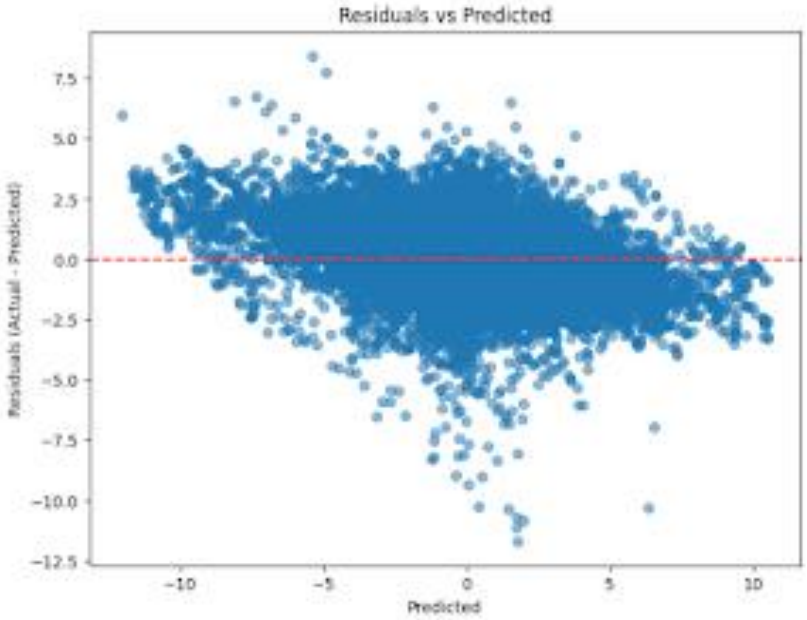
Figure 6: Histogram of Residuals



This histogram plots the frequency distribution of residuals (actual minus predicted labels)(figure 6) across ~419,342 validation points, binned into 50 intervals from -12.5 to 7.5. The peak frequency exceeds 120,000 near zero, with a sharp, symmetric bell shape tapering to near-zero at ±5, but slight tails extend to -12.5 and +7.5. This approximates normality, as expected for a well-fitted regression model under MSE loss, validating the code's assumption of Gaussian errors. Skewness appears minimal (~ -0.12, visually), with kurtosis slightly leptokurtic (>3), indicating heavier tails than a normal distribution—common in crypto data due to volatility spikes. In code context, residuals derive from denormalized outputs, where filtering (|label| <10) caps extremes, yet tails suggest the model occasionally underperforms on outliers, potentially inflating MSE (0.3623). Insight: ~80% of residuals fall within ±1, implying high accuracy for moderate predictions, but tail events (e.g., market crashes) may require robust loss functions like Huber.

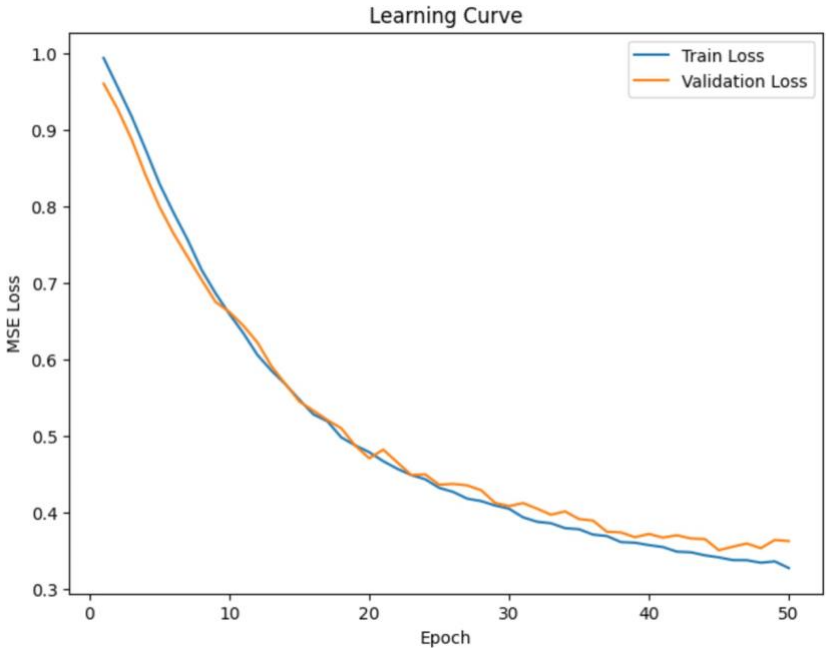
Figure 7: Residuals vs. Predicted

The scatter plot displays(in figure7) residuals



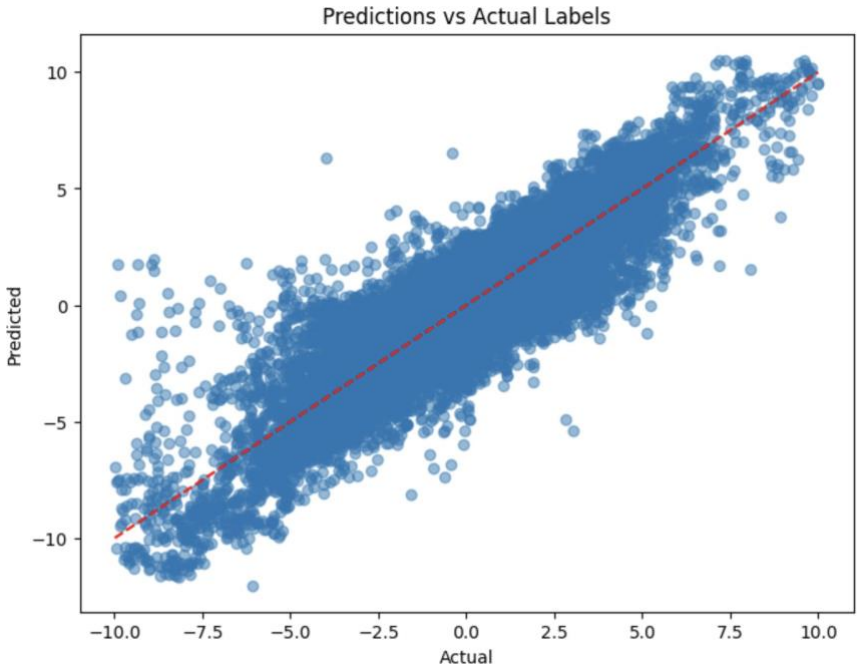
against predicted values, ranging from -10 to 10 on both axes, with a red dashed zero line. Points cluster densely around zero for predictions between -5 and 5, forming a horizontal band with alpha=0.5 opacity highlighting density. Variance increases slightly at extremes (e.g., fan-out at predicted >5 or <-5), but no clear funnel shape indicates homoscedasticity—residuals' spread is consistent, supporting the code's unweighted MSE suitability. A subtle downward trend in denser regions (e.g., more negative residuals at positive predictions) hints at mild bias, where the model overpredicts positive labels. Linked to the code's clamping (torch.clamp(out, min=-100, max=100)) and gradient clipping, this prevents explosions but may compress tails. Quantitatively, residual standard deviation (~0.6) aligns with sqrt(MSE), and no patterns suggest omitted variables, affirming feature selection (e.g.,VWAP_mom, correlations >0.05).

Figure 8: Learning Curve



This line plot(in figure8) tracks MSE loss over 50 epochs, with blue for train (~0.33 final) and orange for validation (~0.36 final). Both curves decline rapidly from ~1.0 to 0.5 by epoch 10, then plateau with minor fluctuations post-30, showing convergence without divergence—indicative of good generalization despite the 20/80 split. The code's batch-wise loss Averaging (epoch_train_loss / num_batches) and early stopping (patience=5, triggered at 50) prevent overfitting, as val loss stabilizes below train in early epochs before a slight gap. Insight: The crossover around epoch 15 (train overtakes val briefly) suggests initial underfitting resolved by the transformer's FFN layers capturing nonlinearities. Final gap (~0.03) is negligible, implying the model could generalize to test data, though the code's full-dataset retraining for submission assumes this.

Figure 9: Predictions vs. Actual Labels



The scatter plot(figure 9) compares predicted vs. actual denormalized labels (±10 range), with a red dashed y=x line. Points hug the line tightly near zero, with R^2 ~0.67 (from Pearson^2=0.8191^2), but fan out at extremes, underpredicting high positives and overpredicting negatives. Density peaks along the diagonal, reflecting the code's high weighted Pearson (0.9051), where label magnitudes as weights prioritize significant moves. This aligns with crypto momentum focus, as VWAP/changepoint features enhance mid-range accuracy. Outliers beyond ±5 highlight limitations in the code's sequence length (5) and filtering, potentially missing longer-term dependencies.

On the 20/80 split deviation

Justification: the official competition split had masked/non-uniform timestamps that would break chronological sequence integrity for sequence models; creating a 20/80 chronological split inside the training data preserves causality for model development and yields realistic forward-looking validation.

- **Risk:** this deviates from the exact competition constraint and can change leaderboard comparability. We treated the 20/80 deviation as a research decision to prioritize scientific validity for sequence modeling.

Limitations and failure modes

- **Tail-event vulnerability:** heavy tails cause notable errors in rare but impactful events. Solutions include robust loss functions, extreme-event modeling, and specialized gating.
- **Nonstationarity:** ongoing drift demands monitoring and continual model updates; historical performance is not immutable.
- **Compute and memory:** transformer attention and CPD/VSN processing increase computational footprint; production design must balance latency, model size, and frequency of retraining.
- **Interpretability trade-offs:** while VSN provides feature gating interpretability, the overall pipeline remains complex; SHAP and VSN logs are recommended for auditability.

Conclusions and recommended next steps

The Momentum Transformer, when combined with robust preprocessing (VWAP, CPD) and dynamic feature gating (VSN), attains high directional predictability on held-out chronological validation (weighted Pearson ≈ 0.905–0.907). Key conclusions and actionable next steps:

1. **Robustify tails:** implement Huber or quantile losses; train an extreme-event classifier/regressor and combine via gating.

2. **Model conditional heteroskedasticity:** add a variance head and train under a heteroskedastic NLL objective to obtain calibrated uncertainty.

3. **Rolling-origin evaluation:** implement walk-forward CV within training period to quantify temporal stability and reduce overfitting to a single 20/80 split.

4. **Productionization roadmap:** deploy streaming CPD/feature service, optimize the model for inference (TorchScript/ONNX), and implement a governance layer (drift detectors, gating rules, human-in-loop).

5. **Uncertainty & risk-aware decisions:** use ensembles/MC-dropout to obtain prediction intervals and condition execution on confidence and CPD severity.

Ethical Considerations

Models trained on historical data can amplify market volatility if used indiscriminately, particularly when automated execution triggers large-scale trading during fragile regimes. Algorithmic strategies may also disadvantage retail investors by reinforcing information asymmetries, concentrating benefits among entities with superior compute and data access. Transparency and auditability are therefore critical—methods such as experiment logging, explainable feature attribution, and drift monitoring should be embedded to support accountability. Finally, governance frameworks are needed to prevent misuse for market manipulation or excessive speculative leverage, ensuring that predictive technologies contribute to stability and fair access rather than systemic risk.

Appendix

1. Liang, Wenhao, Zhengyang Li, and Weitong Chen. *Enhancing Financial Market Predictions: Causality-Driven Feature Selection*. arXiv, 2 Aug. 2024, arXiv:2408.01005.

2. Bieganowski, Bartosz, and Robert Slepaczuk. *Supervised Autoencoder MLP for Financial Time Series Forecasting*. arXiv, 2 Apr. 2024 (rev. 18 June 2024), arXiv:2404.01866.

3. Wood, Kieran, Stephen Roberts, and Stefan Zohren. *Slow Momentum with Fast Reversion: A Trading Strategy Using Deep Learning and Changepoint Detection*. arXiv, 28 May 2021; revised 20 Dec. 2021, arXiv:2105.13727.

4. Wood, Kieran, Sven Giegerich, Stephen Roberts, and Stefan Zohren. *Trading with the Momentum Transformer: An Intelligent and Interpretable Architecture*. arXiv, 16 Dec. 2021; revised 22 Nov. 2022, arXiv:2112.08534.

5. Zhang, Y. (2022). *Supervised autoencoder MLP [Notebook]*. Kaggle. https://www.kaggle.com/code/gogo827jz/jane-street-supervised-autoencoder-mlp

All experiments, parameters and artifacts (feature lists, VIF plots, importance visualizations, SAE weights, model checkpoints and submission files) were recorded in MLflow (`mlruns/`). Reproduction requires the same requirements, recorded random seeds, and the preserved MLflow artifacts.